

EXPERIMENT 5

Student Name: Hunar Katyal

UID: 24BDA70085

Branch: AIT-CSE

Section/Group: 24AIT_KRG-G2

Semester: 5

Subject Name: ADBMS

Subject Code: 24CSP-333

Experiment 5 -- To study and implement advanced SQL queries using Joins, Correlated Query's and Conditional statement.

Aim : To study and implement correlated subqueries in SQL by retrieving employees whose salaries are among the top three highest distinct salary values within their respective departments, while correctly handling duplicate salary values using aggregate functions and joins

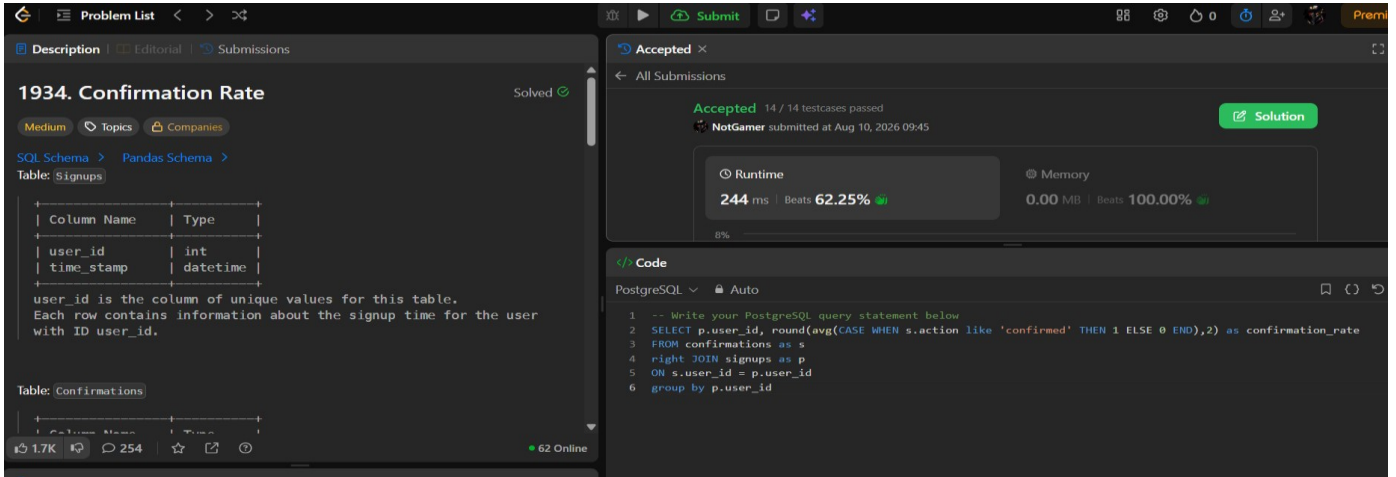
Question :5 Part A (i): Calculate Confirmation Rate

SQL Query:

```
SELECT p.user_id, round(avg(CASE WHEN s.action like 'confirmed' THEN 1 ELSE 0 END),2) as  
confirmation_rate  
FROM confirmations as s  
right JOIN signups as p  
ON s.user_id = p.user_id  
group by p.user_id
```

EXPERIMENT 5

4



The screenshot shows a coding platform interface. On the left, the problem description for '1934. Confirmation Rate' is visible, including a table schema for 'Signups' and 'Confirmations'. The 'Signups' table has columns 'user_id' (int) and 'time_stamp' (datetime). The 'Confirmations' table has columns 'user_id' (int) and 'time_stamp' (datetime). The problem asks to calculate the confirmation rate. On the right, the submission status is 'Accepted', showing 14/14 testcases passed. The runtime is 244 ms, beating 62.25% of solutions. The memory usage is 0.00 MB, beating 100.00% of solutions. The code is written in PostgreSQL and uses a right JOIN to calculate the confirmation rate.

Question: Part A (ii) Calculate Cancellation rate

SQL Query:

```
SELECT tp.request_at as "Day", ROUND(AVG(CASE WHEN tp.status like 'cancelled%' THEN 1 ELSE 0
END),2)as "Cancellation Rate"
```

```
FROM trips as tp
```

```
JOIN Users as c ON c.users_id = tp.client_id
```

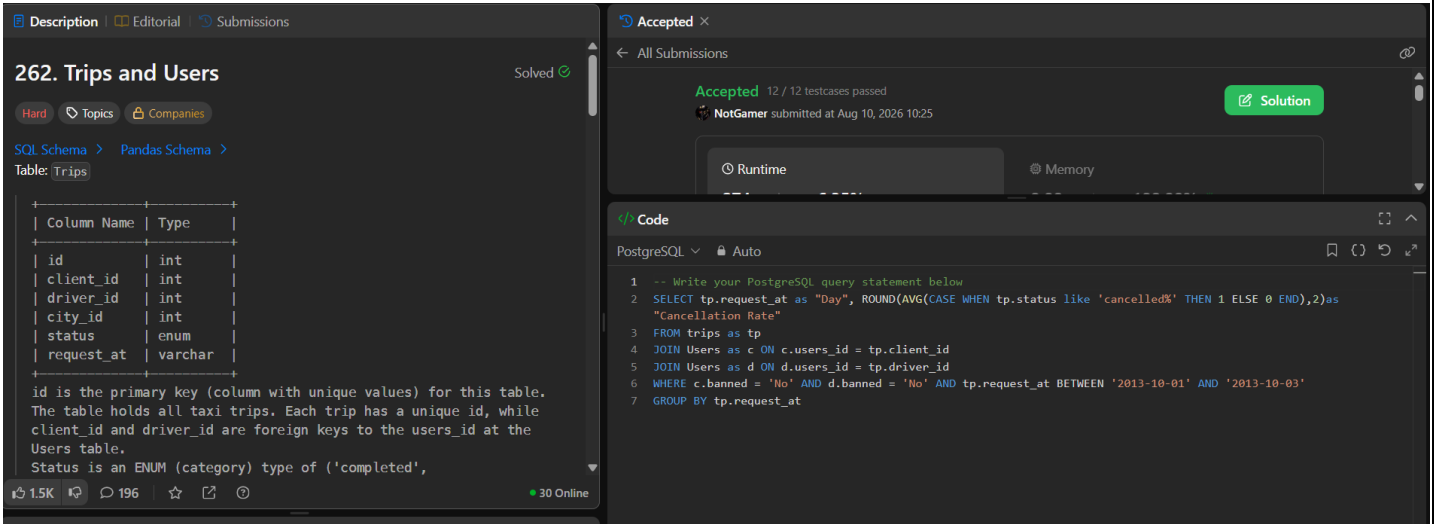
```
JOIN Users as d ON d.users_id = tp.driver_id
```

```
WHERE c.banned = 'No' AND d.banned = 'No' AND tp.request_at BETWEEN '2013-10-01' AND '2013-10-03'
```

```
GROUP BY tp.request_at
```

EXPERIMENT 5

4



The screenshot displays a web-based SQL problem-solving environment. On the left, the problem description for '262. Trips and Users' is shown, including a table schema for 'Trips' and a brief explanation of the data. The table schema is as follows:

Column Name	Type
id	int
client_id	int
driver_id	int
city_id	int
status	enum
request_at	varchar

Below the schema, it states: 'id is the primary key (column with unique values) for this table. The table holds all taxi trips. Each trip has a unique id, while client_id and driver_id are foreign keys to the users_id at the Users table. Status is an ENUM (category) type of ('completed', 'cancelled').'

On the right, the 'Accepted' submission tab is active, showing a solution for the problem. The solution code is a PostgreSQL query that calculates the cancellation rate for each day in 2013, excluding cancelled trips and trips by banned clients or drivers.

```
1 -- Write your PostgreSQL query statement below
2 SELECT tp.request_at as "Day", ROUND(AVG(CASE WHEN tp.status like 'cancelled%' THEN 1 ELSE 0 END),2)as
   "Cancellation Rate"
3 FROM trips as tp
4 JOIN Users as c ON c.users_id = tp.client_id
5 JOIN Users as d ON d.users_id = tp.driver_id
6 WHERE c.banned = 'No' AND d.banned = 'No' AND tp.request_at BETWEEN '2013-10-01' AND '2013-10-03'
7 GROUP BY tp.request_at
```

Learning Outcome

After completing this experiment, the learner will be able to:

1. **Understand and implement correlated subqueries** to solve SQL problems involving row-wise comparisons.
2. **Apply aggregate functions** such as COUNT() and DISTINCT to retrieve top-N records within groups.
3. **Use joins and filtering techniques** to combine data from multiple tables and generate meaningful reports.
4. **Analyze and solve complex SQL queries** involving grouped data, ranking logic, and handling duplicate values effectively.